Penetration Testing Report

Full Name: Ian Joseph
Program: HCPT
Date: 03/03/2025

Introduction

This report document hereby describes the proceedings and results of a Black Box security assessment conducted against the **Week 3 Labs**. The report hereby lists the findings and corresponding best practice mitigation actions and recommendations.

1. Objective

The objective of the assessment was to uncover vulnerabilities in the **Week 3 Labs** and provide a final security assessment report comprising vulnerabilities, remediation strategy and recommendation guidelines to help mitigate the identified vulnerabilities and risks during the activity.

2. Scope

This section defines the scope and boundaries of the project.

Application Name	**Lab 1: Cross-Site Request Forgery** **Lab 2: Cross-Origin Resource Sharing**

3. Summary

Outlined is a Black Box Application Security assessment for the **Week 3 Labs**.

Total number of Sub-labs: 13 Sub-labs

High	Medium	Low
5	4	4

High - **5 Sub-labs with hard difficulty level**

Medium - **4 Sub-labs with medium difficulty level**

Low - **4 Sub-labs with easy difficulty level**

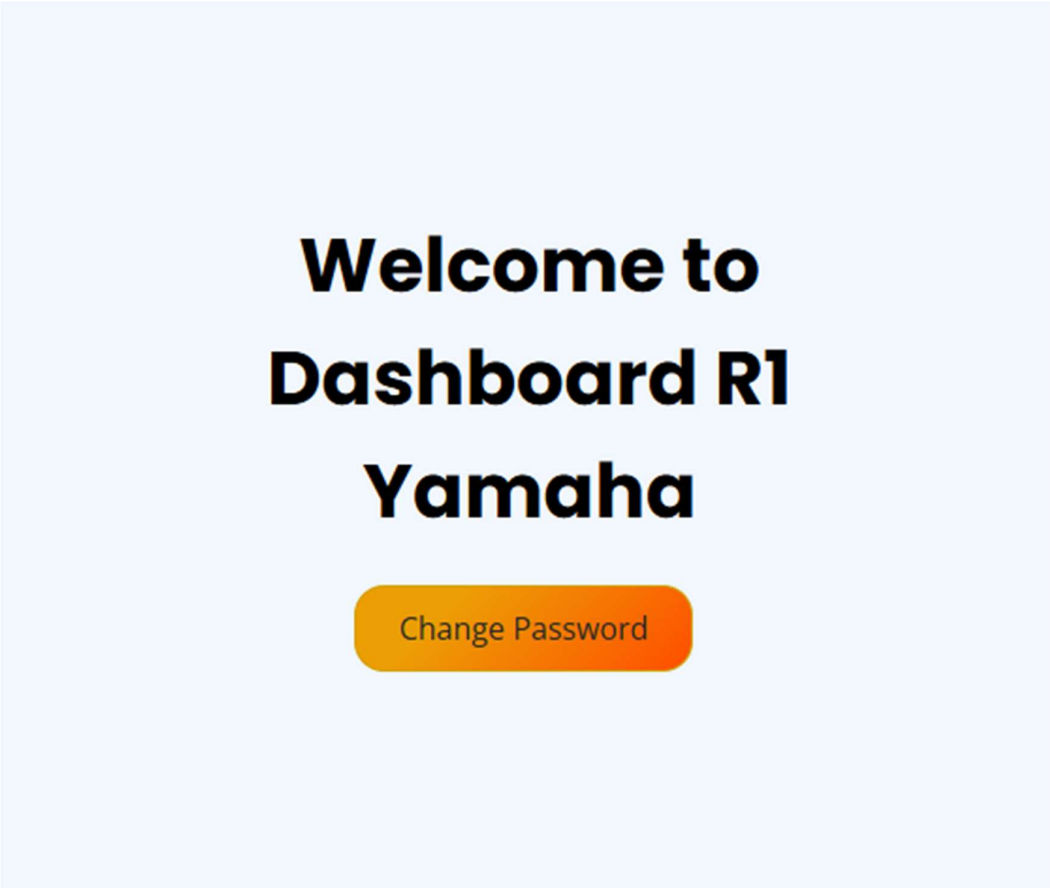
1. Cross-Site Request Forgery

1.1. Eassyy CSRF

Reference	Risk Rating
Eassyy CSRF	**Low**
Tools Used	
Terminal, Burp Suite.	
Vulnerability Description	
A cross-site request forgery (CSRF) attack is a malicious exploit that tricks a user into performing unwanted actions on a website. An attacker sends a forged request to a user's browser The user is already authenticated by the application, so the browser executes the request The attacker can then perform actions like changing passwords, transferring funds, or stealing data	
How It Was Discovered	
Automated Tools / Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/csrf_lab/lab_1/lab_1.php	
Consequences of not Fixing the Issue	
In a successful CSRF attack, the attacker causes the victim user to carry out an action unintentionally. For example, this might be to change the email address on their account, to change their password, or to make a funds transfer. Depending on the nature of the action, the attacker might be able to gain full control over the user's account. If the compromised user has a privileged role within the application, then the attacker might be able to take full control of all the application's data and functionality.	
Suggested Countermeasures	
CSRF tokens - A CSRF token is a unique, secret, and unpredictable value that is generated by the server-side application and shared with the client. When attempting to perform a sensitive action, such as submitting a form, the client must include the correct CSRF token in the request. This makes it very difficult for an attacker to construct a valid request on behalf of the victim. **SameSite cookies** - SameSite is a browser security mechanism that determines when a website's cookies are included in requests originating from other websites. As requests to perform sensitive actions typically require an authenticated session cookie, the appropriate SameSite restrictions may prevent an attacker from triggering these actions cross-site. Since 2021, Chrome enforces Lax SameSite restrictions by default. As this is the proposed standard, we expect other major browsers to adopt this behavior in future. **Referer-based validation** - Some applications make use of the HTTP Referer header to attempt to defend against CSRF attacks, normally by verifying that the request originated from the application's own domain. This is generally less effective than CSRF token validation.	
References	
https://portswigger.net/web-security/csrf https://owasp.org/www-community/attacks/csrf https://www.blackduck.com/glossary/what-is-csrf.html https://brightsec.com/blog/cross-site-request-forgery-csrf/ https://www.cloudflare.com/learning/security/threats/cross-site-request-forgery/	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



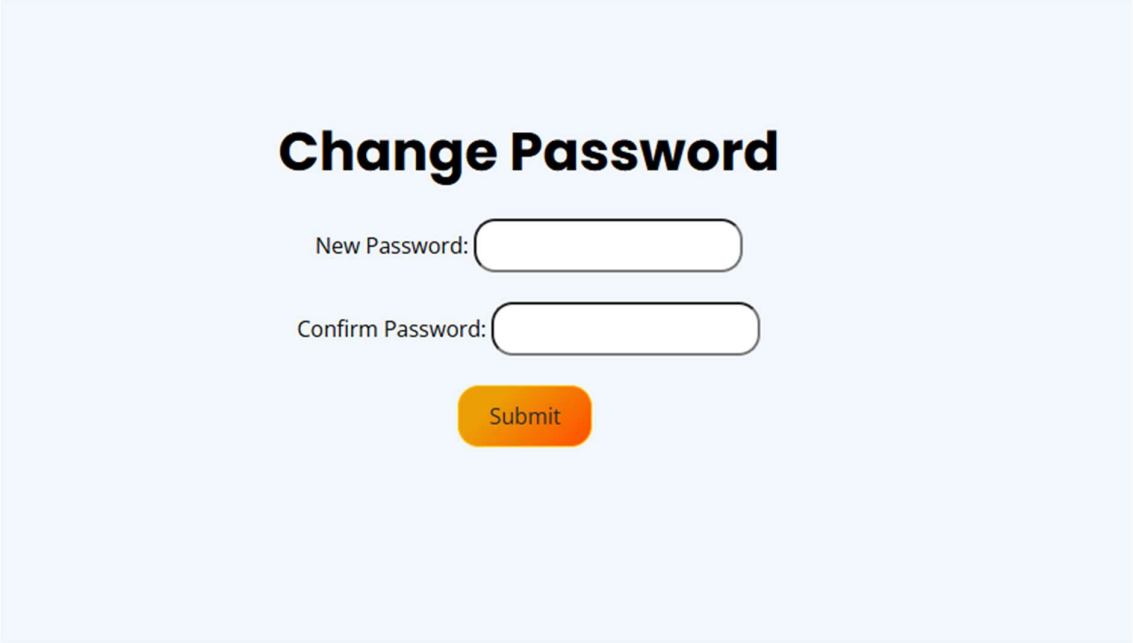
1.2. Always Validate Tokens

Reference	Risk Rating
Always Validate Tokens	**Medium**
Tools Used	
Terminal and Burp Suite.	
Vulnerability Description	
A cross-site request forgery (CSRF) attack is a malicious exploit that tricks a user into performing unwanted actions on a website. An attacker sends a forged request to a user's browser The user is already authenticated by the application, so the browser executes the request The attacker can then perform actions like changing passwords, transferring funds, or stealing data	
How It Was Discovered	
Automated Tools / Manual Analysis	
Vulnerable URLs	

https://labs.hacktify.in/HTML/csrf_lab/lab_2/lab_2.php
Consequences of not Fixing the Issue
In a successful CSRF attack, the attacker causes the victim user to carry out an action unintentionally. For example, this might be to change the email address on their account, to change their password, or to make a funds transfer. Depending on the nature of the action, the attacker might be able to gain full control over the user's account. If the compromised user has a privileged role within the application, then the attacker might be able to take full control of all the application's data and functionality.
Suggested Countermeasures
CSRF tokens - A CSRF token is a unique, secret, and unpredictable value that is generated by the server-side application and shared with the client. When attempting to perform a sensitive action, such as submitting a form, the client must include the correct CSRF token in the request. This makes it very difficult for an attacker to construct a valid request on behalf of the victim. SameSite cookies - SameSite is a browser security mechanism that determines when a website's cookies are included in requests originating from other websites. As requests to perform sensitive actions typically require an authenticated session cookie, the appropriate SameSite restrictions may prevent an attacker from triggering these actions cross-site. Since 2021, Chrome enforces Lax SameSite restrictions by default. As this is the proposed standard, we expect other major browsers to adopt this behavior in future. Referer-based validation - Some applications make use of the HTTP Referer header to attempt to defend against CSRF attacks, normally by verifying that the request originated from the application's own domain. This is generally less effective than CSRF token validation.
References
https://portswigger.net/web-security/csrf https://owasp.org/www-community/attacks/csrf https://www.blackduck.com/glossary/what-is-csrf.html https://brightsec.com/blog/cross-site-request-forgery-csrf/ https://www.cloudflare.com/learning/security/threats/cross-site-request-forgery/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



1.3. I hate when someone uses my tokens!

Reference	Risk Rating
I hate when someone uses my tokens!	**Medium**
Tools Used	
Terminal, Burp Suite.	
Vulnerability Description	
A cross-site request forgery (CSRF) attack is a malicious exploit that tricks a user into performing unwanted actions on a website. An attacker sends a forged request to a user's browser The user is already authenticated by the application, so the browser executes the request The attacker can then perform actions like changing passwords, transferring funds, or stealing data	
How It Was Discovered	
Automated Tools / Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/csrf_lab/lab_4/lab_4.php	
Consequences of not Fixing the Issue	
In a successful CSRF attack, the attacker causes the victim user to carry out an action unintentionally. For example, this might be to change the email address on their account, to change their password, or to make a funds transfer. Depending on the nature of the action, the attacker might be able to gain full control over the user's account. If the compromised user has a privileged role within the application, then the attacker might be able to take full control of all the application's data and functionality.	
Suggested Countermeasures	
CSRF tokens - A CSRF token is a unique, secret, and unpredictable value that is generated by the server-side application and shared with the client. When attempting to perform a sensitive action, such as submitting a form, the client must include the correct CSRF token in the request. This makes it very difficult for an attacker to construct a valid request on behalf of the victim.	

SameSite cookies - SameSite is a browser security mechanism that determines when a website's cookies are included in requests originating from other websites. As requests to perform sensitive actions typically require an authenticated session cookie, the appropriate SameSite restrictions may prevent an attacker from triggering these actions cross-site. Since 2021, Chrome enforces Lax SameSite restrictions by default. As this is the proposed standard, we expect other major browsers to adopt this behavior in future.

Referer-based validation - Some applications make use of the HTTP Referer header to attempt to defend against CSRF attacks, normally by verifying that the request originated from the application's own domain. This is generally less effective than CSRF token validation.

References

https://portswigger.net/web-security/csrf
https://owasp.org/www-community/attacks/csrf
https://www.blackduck.com/glossary/what-is-csrf.html
https://brightsec.com/blog/cross-site-request-forgery-csrf/
https://www.cloudflare.com/learning/security/threats/cross-site-request-forgery/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

1.4. GET Me or POST ME

Reference	Risk Rating
GET Me or POST ME	**Low**
Tools Used	
Terminal, Burp Suite.	
Vulnerability Description	
A cross-site request forgery (CSRF) attack is a malicious exploit that tricks a user into performing unwanted actions on a website. An attacker sends a forged request to a user's browser The user is already authenticated by the application, so the browser executes the request The attacker can then perform actions like changing passwords, transferring funds, or stealing data	
How It Was Discovered	
Automated Tools / Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/csrf_lab/lab_6/lab_6.php	
Consequences of not Fixing the Issue	
In a successful CSRF attack, the attacker causes the victim user to carry out an action unintentionally. For example, this might be to change the email address on their account, to change their password, or to make a funds transfer. Depending on the nature of the action, the attacker might be able to gain full control over the user's account. If the compromised user has a privileged role within the application, then the attacker might be able to take full control of all the application's data and functionality.	
Suggested Countermeasures	

CSRF tokens - A CSRF token is a unique, secret, and unpredictable value that is generated by the server-side application and shared with the client. When attempting to perform a sensitive action, such as submitting a form, the client must include the correct CSRF token in the request. This makes it very difficult for an attacker to construct a valid request on behalf of the victim.
SameSite cookies - SameSite is a browser security mechanism that determines when a website's cookies are included in requests originating from other websites. As requests to perform sensitive actions typically require an authenticated session cookie, the appropriate SameSite restrictions may prevent an attacker from triggering these actions cross-site. Since 2021, Chrome enforces Lax SameSite restrictions by default. As this is the proposed standard, we expect other major browsers to adopt this behavior in future.
Referer-based validation - Some applications make use of the HTTP Referer header to attempt to defend against CSRF attacks, normally by verifying that the request originated from the application's own domain. This is generally less effective than CSRF token validation.

References

https://portswigger.net/web-security/csrf
https://owasp.org/www-community/attacks/csrf
https://www.blackduck.com/glossary/what-is-csrf.html
https://brightsec.com/blog/cross-site-request-forgery-csrf/
https://www.cloudflare.com/learning/security/threats/cross-site-request-forgery/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

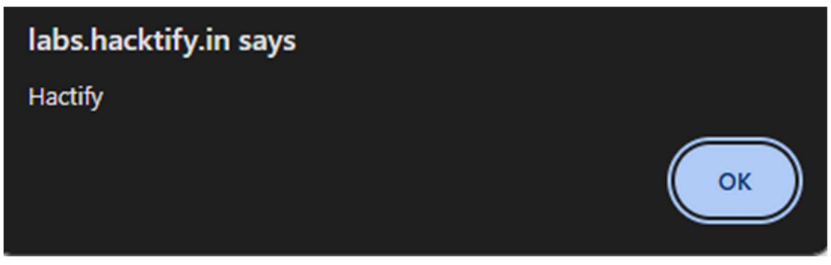
1.5. XSS the saviour

Reference	Risk Rating
XSS the saviour	**Hard**
Tools Used	
Terminal, Burp Suite.	
Vulnerability Description	
A cross-site request forgery (CSRF) attack is a malicious exploit that tricks a user into performing unwanted actions on a website. An attacker sends a forged request to a user's browser The user is already authenticated by the application, so the browser executes the request The attacker can then perform actions like changing passwords, transferring funds, or stealing data	
How It Was Discovered	
Automated Tools / Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/csrf_lab/lab_7/lab_7.php	
Consequences of not Fixing the Issue	
In a successful CSRF attack, the attacker causes the victim user to carry out an action unintentionally. For example, this might be to change the email address on their account, to change their password, or to make a funds transfer. Depending on the nature of the action, the attacker might be able to gain full	

control over the user's account. If the compromised user has a privileged role within the application, then the attacker might be able to take full control of all the application's data and functionality.
Suggested Countermeasures
CSRF tokens - A CSRF token is a unique, secret, and unpredictable value that is generated by the server-side application and shared with the client. When attempting to perform a sensitive action, such as submitting a form, the client must include the correct CSRF token in the request. This makes it very difficult for an attacker to construct a valid request on behalf of the victim. **SameSite cookies** - SameSite is a browser security mechanism that determines when a website's cookies are included in requests originating from other websites. As requests to perform sensitive actions typically require an authenticated session cookie, the appropriate SameSite restrictions may prevent an attacker from triggering these actions cross-site. Since 2021, Chrome enforces Lax SameSite restrictions by default. As this is the proposed standard, we expect other major browsers to adopt this behavior in future. **Referer-based validation** - Some applications make use of the HTTP Referer header to attempt to defend against CSRF attacks, normally by verifying that the request originated from the application's own domain. This is generally less effective than CSRF token validation.
References
https://portswigger.net/web-security/csrf https://owasp.org/www-community/attacks/csrf https://www.blackduck.com/glossary/what-is-csrf.html https://brightsec.com/blog/cross-site-request-forgery-csrf/ https://www.cloudflare.com/learning/security/threats/cross-site-request-forgery/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



1.6. rm -rf token

Reference	Risk Rating
rm -rf token	**Hard**
Tools Used	
Terminal, Burp Suite.	
Vulnerability Description	

A cross-site request forgery (CSRF) attack is a malicious exploit that tricks a user into performing unwanted actions on a website. An attacker sends a forged request to a user's browser The user is already authenticated by the application, so the browser executes the request The attacker can then perform actions like changing passwords, transferring funds, or stealing data
How It Was Discovered
Automated Tools / Manual Analysis
Vulnerable URLs
https://labs.hacktify.in/HTML/csrf_lab/lab_8/lab_8.php
Consequences of not Fixing the Issue
In a successful CSRF attack, the attacker causes the victim user to carry out an action unintentionally. For example, this might be to change the email address on their account, to change their password, or to make a funds transfer. Depending on the nature of the action, the attacker might be able to gain full control over the user's account. If the compromised user has a privileged role within the application, then the attacker might be able to take full control of all the application's data and functionality.
Suggested Countermeasures
CSRF tokens - A CSRF token is a unique, secret, and unpredictable value that is generated by the server-side application and shared with the client. When attempting to perform a sensitive action, such as submitting a form, the client must include the correct CSRF token in the request. This makes it very difficult for an attacker to construct a valid request on behalf of the victim. **SameSite cookies** - SameSite is a browser security mechanism that determines when a website's cookies are included in requests originating from other websites. As requests to perform sensitive actions typically require an authenticated session cookie, the appropriate SameSite restrictions may prevent an attacker from triggering these actions cross-site. Since 2021, Chrome enforces Lax SameSite restrictions by default. As this is the proposed standard, we expect other major browsers to adopt this behavior in future. **Referer-based validation** - Some applications make use of the HTTP Referer header to attempt to defend against CSRF attacks, normally by verifying that the request originated from the application's own domain. This is generally less effective than CSRF token validation.
References
https://portswigger.net/web-security/csrf https://owasp.org/www-community/attacks/csrf https://www.blackduck.com/glossary/what-is-csrf.html https://brightsec.com/blog/cross-site-request-forgery-csrf/ https://www.cloudflare.com/learning/security/threats/cross-site-request-forgery/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

2. Cross-Origin Resource Sharing

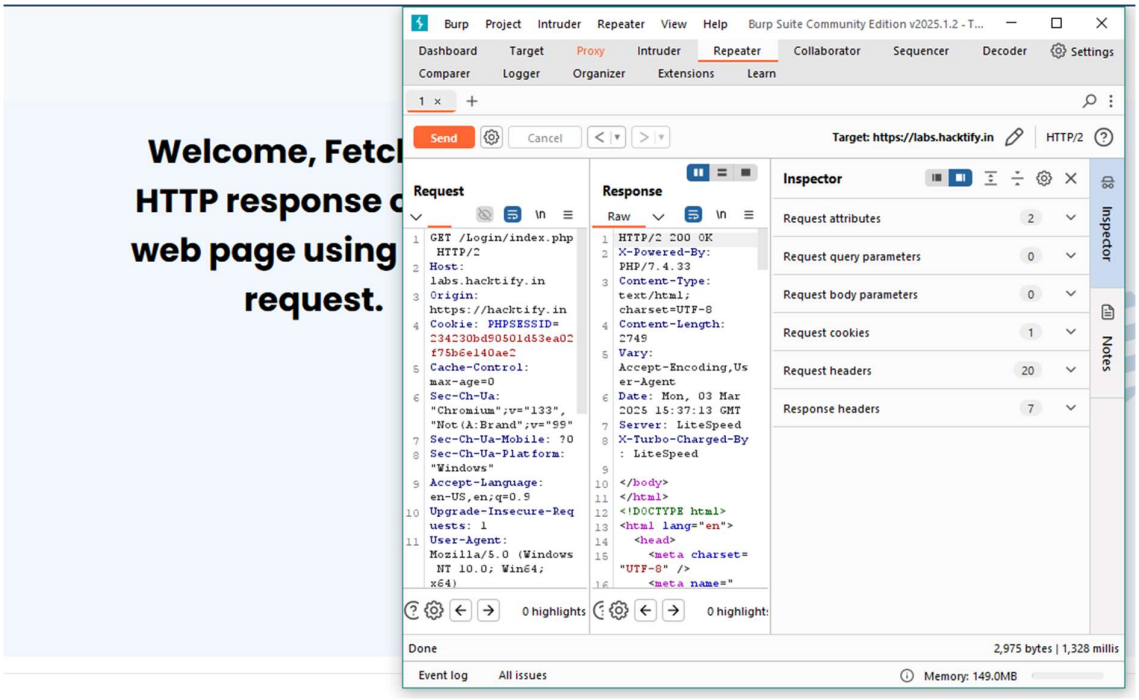
2.1. CORS With Arbitrary Origin

Reference	Risk Rating
CORS With Arbitrary Origin	**Low**
Tools Used	
Terminal, Burp Suite	
Vulnerability Description	
CORS vulnerabilities occur when a web application allows cross-domain requests without proper validation of the origin.	
How It Was Discovered	
Automated Tools / Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/cors_lab/lab_1/cors_1.php	
Consequences of not Fixing the Issue	
This can lead to an attacker being able to perform unauthorized actions, such as stealing sensitive information, through malicious JavaScript code executed on the victim's browser.	
Suggested Countermeasures	
Whitelisting trusted origins Restricting HTTP methods	

Adding custom headers Using preflight requests
References
https://medium.com/@tushar_rs_/cross-origin-resource-sharing-cors-vulnerability-example-and-prevention-588d299ff185 https://portswigger.net/web-security/cors https://www.vaadata.com/blog/understanding-and-preventing-cors-misconfiguration/ https://www.packetlabs.net/posts/cross-origin-resource-sharing-cors

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



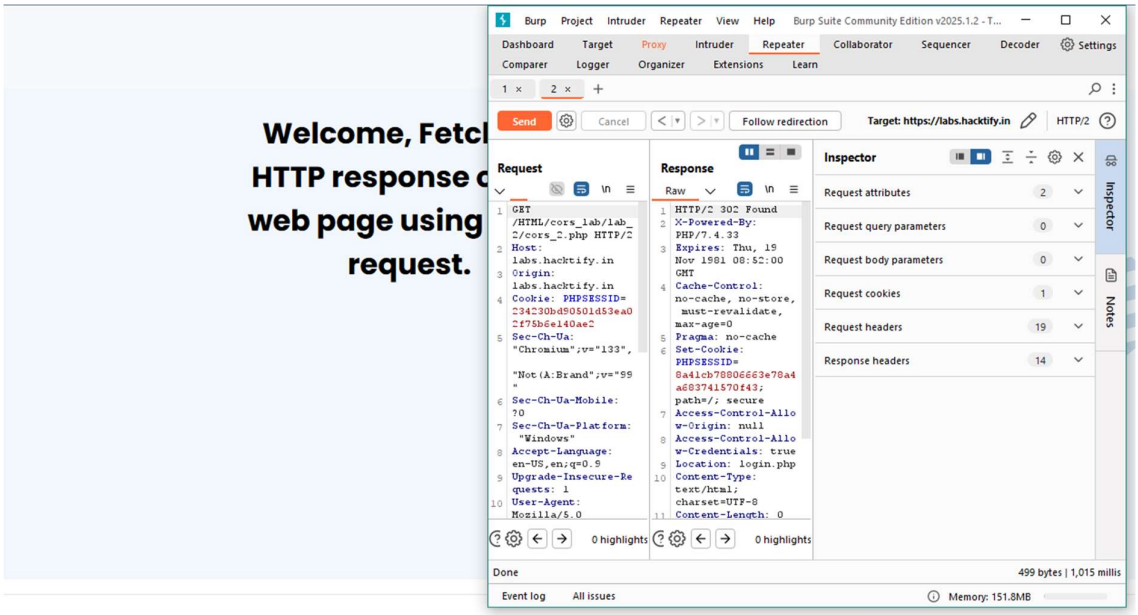
2.2. CORS with Null origin

Reference	Risk Rating
CORS with Null origin	**Low**
Tools Used	
Terminal, Burp Suite	
Vulnerability Description	
CORS vulnerabilities occur when a web application allows cross-domain requests without proper validation of the origin.	
How It Was Discovered	
Automated Tools / Manual Analysis	

Vulnerable URLs
https://labs.hacktify.in/HTML/cors_lab/lab_2/cors_2.php
Consequences of not Fixing the Issue
This can lead to an attacker being able to perform unauthorized actions, such as stealing sensitive information, through malicious JavaScript code executed on the victim's browser.
Suggested Countermeasures
Whitelisting trusted origins Restricting HTTP methods **Adding custom headers** **Using preflight requests**
References
https://medium.com/@tushar_rs_/cross-origin-resource-sharing-cors-vulnerability-example-and-prevention-588d299ff185 https://portswigger.net/web-security/cors https://www.vaadata.com/blog/understanding-and-preventing-cors-misconfiguration/ https://www.packetlabs.net/posts/cross-origin-resource-sharing-cors

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



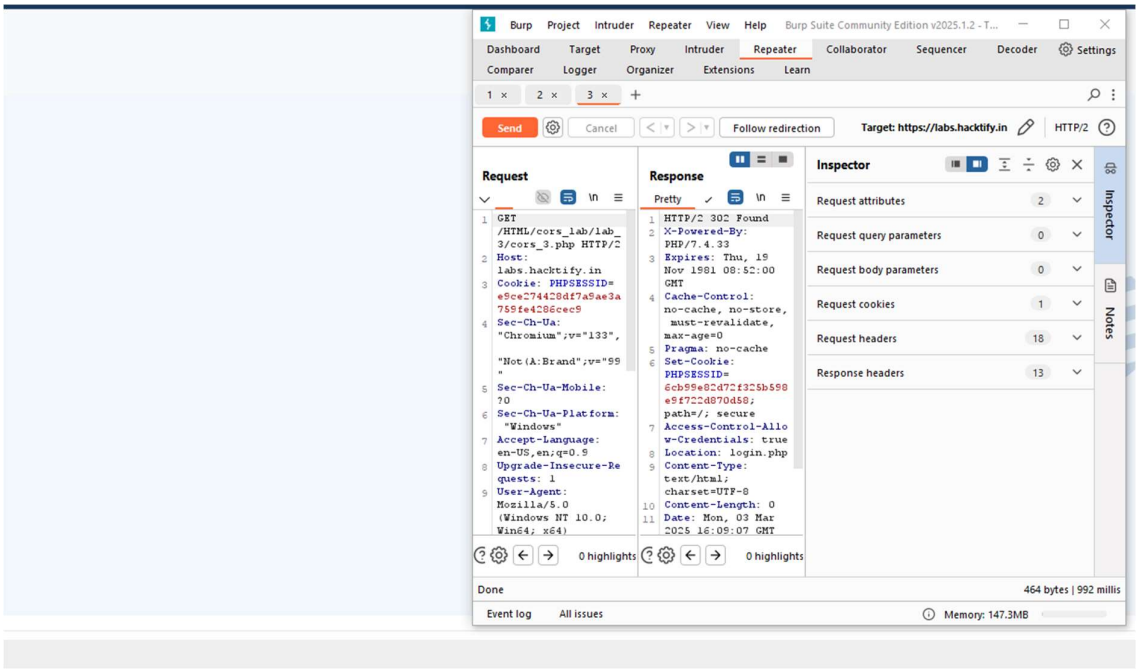
2.3. CORS with prefix match

Reference	Risk Rating

CORS with prefix match	**Medium**
Tools Used	
Terminal, Burp Suite	
Vulnerability Description	
CORS vulnerabilities occur when a web application allows cross-domain requests without proper validation of the origin.	
How It Was Discovered	
Automated Tools / Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/cors_lab/lab_3/cors_3.php	
Consequences of not Fixing the Issue	
This can lead to an attacker being able to perform unauthorized actions, such as stealing sensitive information, through malicious JavaScript code executed on the victim's browser.	
Suggested Countermeasures	
Whitelisting trusted origins Restricting HTTP methods **Adding custom headers** **Using preflight requests**	
References	
https://medium.com/@tushar_rs_/cross-origin-resource-sharing-cors-vulnerability-example-and-prevention-588d299ff185 https://portswigger.net/web-security/cors https://www.vaadata.com/blog/understanding-and-preventing-cors-misconfiguration/ https://www.packetlabs.net/posts/cross-origin-resource-sharing-cors	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



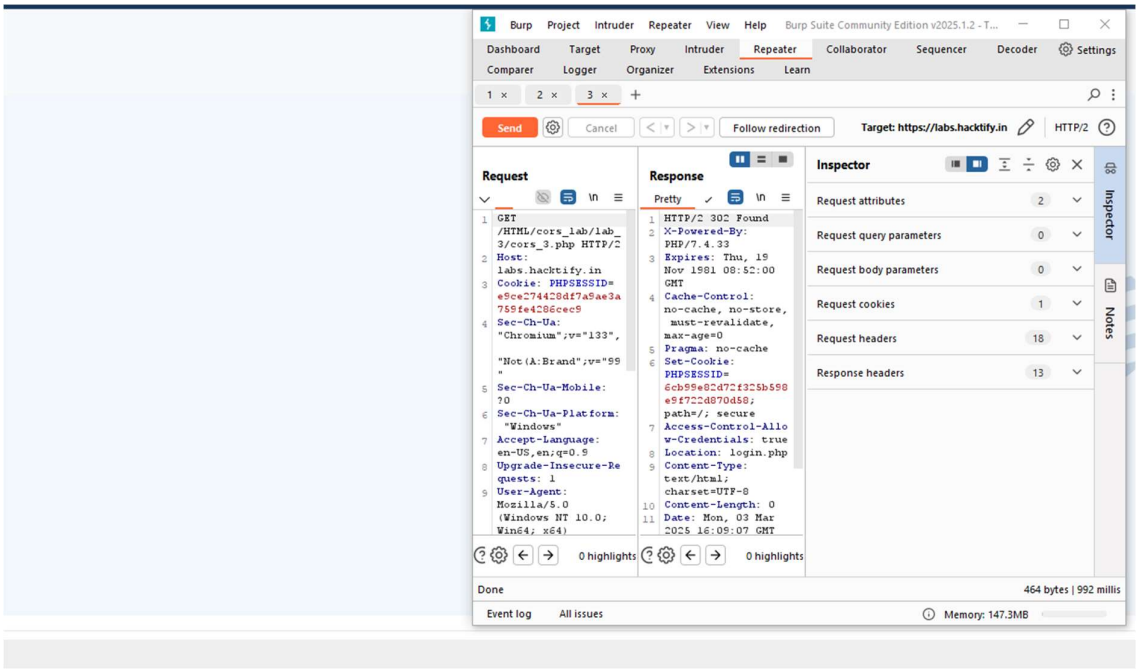
2.4. CORS with suffix match

Reference	Risk Rating
CORS with suffix match	**Medium**
Tools Used	
Terminal, Burp Suite	
Vulnerability Description	
CORS vulnerabilities occur when a web application allows cross-domain requests without proper validation of the origin.	
How It Was Discovered	
Automated Tools / Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/cors_lab/lab_4/cors_4.php	
Consequences of not Fixing the Issue	
This can lead to an attacker being able to perform unauthorized actions, such as stealing sensitive information, through malicious JavaScript code executed on the victim's browser.	
Suggested Countermeasures	
Whitelisting trusted origins Restricting HTTP methods **Adding custom headers** **Using preflight requests**	
References	
https://medium.com/@tushar_rs_/cross-origin-resource-sharing-cors-vulnerability-example-and-prevention-588d299ff185 https://portswigger.net/web-security/cors https://www.vaadata.com/blog/understanding-and-preventing-cors-misconfiguration/	

https://www.packetlabs.net/posts/cross-origin-resource-sharing-cors

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



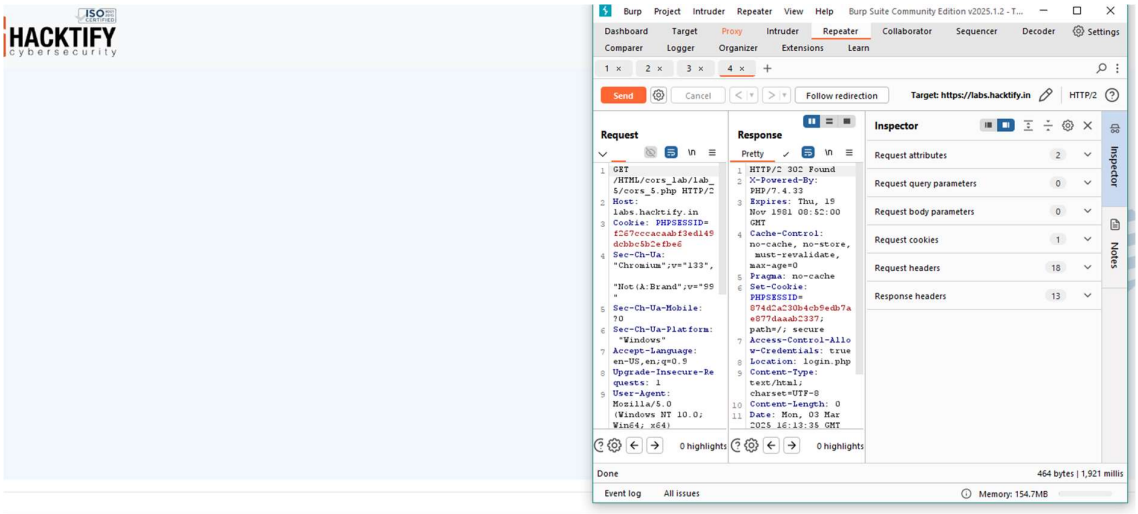
2.5. CORS with Escape dot

Reference	Risk Rating
CORS with Escape dot	**Hard**
Tools Used	
Terminal, Burp Suite	
Vulnerability Description	
CORS vulnerabilities occur when a web application allows cross-domain requests without proper validation of the origin.	
How It Was Discovered	
Automated Tools / Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/cors_lab/lab_5/cors_5.php	
Consequences of not Fixing the Issue	
This can lead to an attacker being able to perform unauthorized actions, such as stealing sensitive information, through malicious JavaScript code executed on the victim's browser.	
Suggested Countermeasures	
Whitelisting trusted origins Restricting HTTP methods	

Adding custom headers **Using preflight requests**
References
https://medium.com/@tushar_rs_/cross-origin-resource-sharing-cors-vulnerability-example-and-prevention-588d299ff185 https://portswigger.net/web-security/cors https://www.vaadata.com/blog/understanding-and-preventing-cors-misconfiguration/ https://www.packetlabs.net/posts/cross-origin-resource-sharing-cors

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



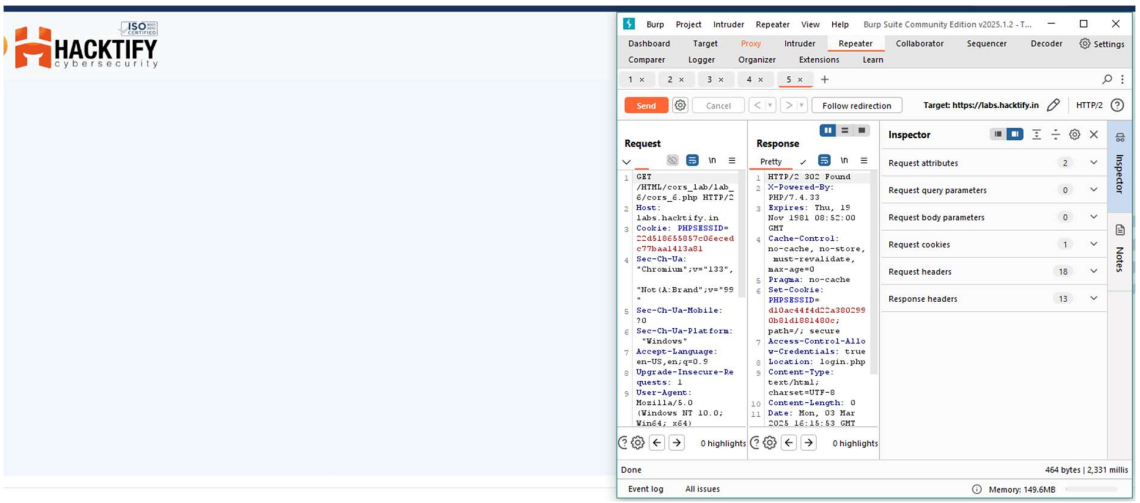
2.6. CORS with Substring match

Reference	**Risk Rating**
CORS with Substring match	**Hard**
Tools Used	
Terminal, Burp Suite	
Vulnerability Description	
CORS vulnerabilities occur when a web application allows cross-domain requests without proper validation of the origin.	
How It Was Discovered	
Automated Tools / Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/cors_lab/lab_6/cors_6.php	
Consequences of not Fixing the Issue	
This can lead to an attacker being able to perform unauthorized actions, such as stealing sensitive information, through malicious JavaScript code executed on the victim's browser.	

Suggested Countermeasures
Whitelisting trusted origins Restricting HTTP methods **Adding custom headers** **Using preflight requests**
References
https://medium.com/@tushar_rs_/cross-origin-resource-sharing-cors-vulnerability-example-and-prevention-588d299ff185 https://portswigger.net/web-security/cors https://www.vaadata.com/blog/understanding-and-preventing-cors-misconfiguration/ https://www.packetlabs.net/posts/cross-origin-resource-sharing-cors

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



2.7. CORS with Arbitrary Subdomain

Reference	Risk Rating
CORS with Arbitrary Subdomain	**Hard**
Tools Used	
Terminal, Burp Suite	
Vulnerability Description	
CORS vulnerabilities occur when a web application allows cross-domain requests without proper validation of the origin.	
How It Was Discovered	
Automated Tools / Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/cors_lab/lab_7/cors_7.php	

Consequences of not Fixing the Issue
This can lead to an attacker being able to perform unauthorized actions, such as stealing sensitive information, through malicious JavaScript code executed on the victim's browser.
Suggested Countermeasures
Whitelisting trusted origins Restricting HTTP methods **Adding custom headers** **Using preflight requests**
References
https://medium.com/@tushar_rs_/cross-origin-resource-sharing-cors-vulnerability-example-and-prevention-588d299ff185 https://portswigger.net/web-security/cors https://www.vaadata.com/blog/understanding-and-preventing-cors-misconfiguration/ https://www.packetlabs.net/posts/cross-origin-resource-sharing-cors

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab